KJ's Educational Institute
TRINITY COLLEGE OF ENGINEERING & RESEARCH, PUNE
(Approved by AICTE, Affiliated to SPPU)
(Accredited by NAAC with 'A+' Grade)

# TE IT

# AIML

# LAB MANUAL

# SEM 1

# AY 2026-27

**Savitribai Phule Pune University, Pune**
**Third Year Information Technology (2024 Course)**

# PCC-305-ITT: Artificial Intelligence & Machine Learning Lab 2

## Course Objectives:

1. Develop practical skills in implementing Artificial Intelligence techniques for problem-solving and decision-making.
2. Apply search, reasoning and knowledge representation methods to solve real-world AI problems.
3. Perform data preprocessing, visualization and supervised machine learning for predictive analytics.
4. Implement unsupervised and advanced machine learning algorithms for pattern discovery and model improvement.
5. Design and evaluate AI-based solutions using appropriate tools, performance metrics and optimization techniques.

## Course Outcomes:

On completion of the course, students will be able to

**CO1:** Implement intelligent agents, search algorithms and knowledge representation techniques for AI-based problem solving.

**CO2:** Apply data preprocessing, exploratory data analysis and supervised learning algorithms for prediction and classification tasks.

**CO3:** Develop and evaluate machine learning models using appropriate performance metrics and validation techniques.

**CO4:** Implement clustering, association analysis and dimensionality reduction techniques for pattern discovery in datasets.

**CO5:** Build optimized AI/ML solutions using advanced learning algorithms and apply them to real-world applications.

| Assignment No. | Name of the Assignment /Experiment |
|---|---|
| 1 | Design and implement a simple intelligent agent for a real-world application such as a chatbot, smart home controller or recommendation assistant. Demonstrate the PEAS (Performance Measure, Environment, Actuators, Sensors) representation and agent behaviour. |
| 2 | Implement Breadth First Search (BFS) and Depth First Search (DFS) algorithms for solving route-finding or maze navigation problems. Compare their performance based on path length, nodes explored and execution time**.** |
| 3 | Develop a rule-based expert system using propositional logic and implement Forward Chaining and Backward Chaining mechanisms for intelligent decision making. |
| 4 | Download a standard dataset (Heart Disease, Diabetes or Titanic dataset) and perform:<br>Data cleaning<br>Missing value handling<br>Feature scaling<br>Encoding techniques<br>Statistical analysis<br>Data visualization using histograms and boxplots<br>Split the dataset into training and testing sets. |
| 5 | Implement Linear Regression and Polynomial Regression on a suitable dataset such as California Housing or House Price Prediction dataset.<br>Evaluate model performance using:<br>• Mean Squared Error (MSE)<br>• Mean Absolute Error (MAE)<br>• $R^2$ Score<br>Compare the results obtained by both techniques. |
| 6 | Implement Decision Tree and Naïve Bayes classifiers on a suitable dataset.<br>Compare classifier performance using standard evaluation metrics and identify the most suitable model for the selected dataset. |
| **7** | Implement:<br>• K-Means Clustering<br>• Hierarchical Clustering<br>Use the Elbow Method and Dendrogram for determining the optimal number of clusters.<br>Analyze the generated clusters using a real-world dataset. |
| 8 | Apply PCA on a high-dimensional dataset.<br>Compare:<br>• Number of features before and after PCA<br>• Visualization of reduced dimensions<br>• Impact on classification performance |

| | |
|---|---|
| 9 | Implement and compare:<br>• Support Vector Machine (SVM)<br>• Random Forest<br>• AdaBoost<br><br>Perform:<br>• Cross Validation<br>• Hyperparameter Tuning using Grid Search<br><br>Evaluate and compare model performance before and after optimization. |
| 10 | **AI/ML-Based Real-World Application (CO1–CO5)**<br>• Disease Prediction System<br>• Fraud Detection System<br>• Customer Segmentation<br>• Recommendation System<br>• Student Performance Prediction<br>• Crop Yield Prediction<br>**The project should include:**<br>• Problem Definition<br>• Dataset Collection<br>• Data Preprocessing<br>• Model Development<br>• Performance Evaluation<br>• Model Optimization<br>• Result Analysis and Documentation |

# Assignment No: 01

## PR1: Intelligent Agents and PEAS Representation (CO1)

Design and implement a simple intelligent agent for a real-world application such as a chatbot, smart home controller or recommendation assistant. Demonstrate the PEAS (Performance Measure, Environment, Actuators, Sensors) representation and agent behaviour.

**Aim**

To design and implement a simple intelligent agent for a real-world application (Chatbot Assistant) and demonstrate the **PEAS (Performance Measure, Environment, Actuators, Sensors)** representation and intelligent agent behaviour.

**Theory**

**Intelligent Agent**

An **Intelligent Agent** is a software program that can perceive its environment using sensors, process the received information, make decisions, and perform actions using actuators to achieve a specific goal.

The agent continuously follows the cycle:

**Sense → Think → Act**

Example:

- Chatbot
- Smart Home Controller
- Recommendation System
- Self-driving Car
- Virtual Assistant

**Chatbot Agent**

A **Chatbot Agent** is an intelligent software application that interacts with users through text or voice. It receives user input, understands the request, and generates an appropriate response.

Example:

User: Hello
Chatbot: Hi! How can I help you?
User: What is AI?
Chatbot: AI stands for Artificial Intelligence.

**PEAS Representation**

| Component | Description |
|---|---|
| **Performance Measure** | Provide accurate, relevant, and helpful responses quickly. |
| **Environment** | User interacting through a computer or mobile keyboard. |
| **Actuators** | Display chatbot responses on the screen. |
| **Sensors** | Read text entered by the user through the keyboard. |

**Agent Behaviour**

The chatbot follows these steps:

1. Wait for user input.
2. Read the message.

3. Compare it with predefined questions.
4. Generate the appropriate response.
5. Display the response.
6. Continue until the user types **exit**.

# Algorithm

**Step 1:** Start the program.

**Step 2:** Display the PEAS representation.

**Step 3:** Wait for user input.

**Step 4:** Read the user's message.

**Step 5:** Convert the input into lowercase.

**Step 6:** Check whether the input matches predefined keywords.

**Step 7:** If a match is found, display the corresponding response.

**Step 8:** Otherwise, display "Sorry, I don't understand."

**Step 9:** Repeat Steps 3–8 until the user enter exit**.**

**Step 10:** Stop the program.

**Pseudocode**

```
START

Display "Chatbot Intelligent Agent"

Display PEAS Representation

Repeat

   Read user message

   Convert message to lowercase

   IF message = "hello"
      Display "Hello! How can I help you?"

   ELSE IF message = "how are you"
      Display "I am fine."

   ELSE IF message = "what is ai"
      Display "Artificial Intelligence is the simulation of human intelligence."
```

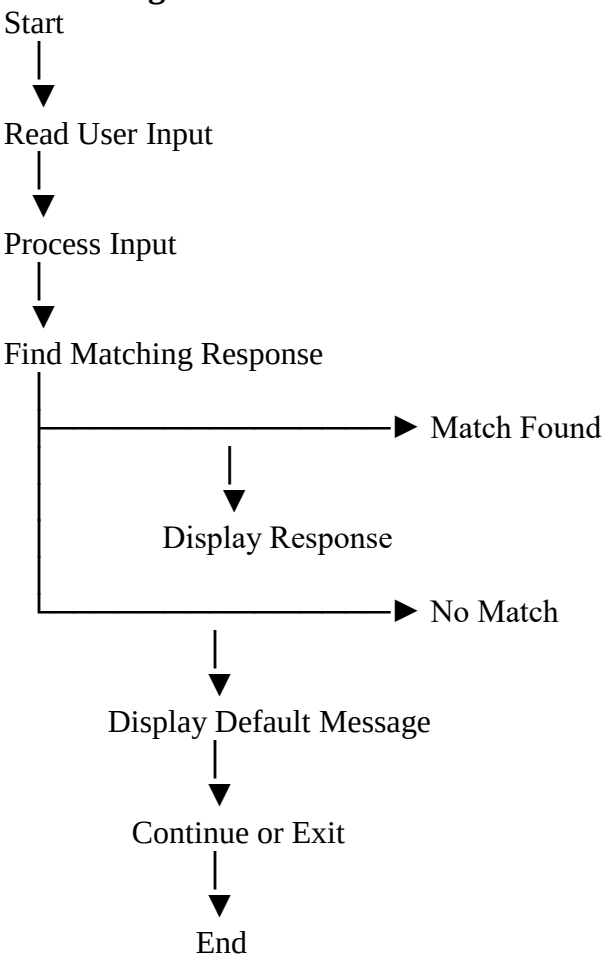
```
    ELSE IF message = "bye" OR "exit"
        Display "Goodbye!"
        Exit loop

    ELSE
        Display "Sorry, I don't understand."

Until user enters exit

END
```

**Flow of Agent**




```
# ------------------------------------------------------------
# AI & ML Laboratory Practical
# Title : Design and Implement a Simple Intelligent Agent
# Application : Chatbot using PEAS Representation
# ------------------------------------------------------------

# Create a class named ChatbotAgent
class ChatbotAgent:
```

```
# Constructor method (automatically executed when object is created)
def __init__(self):

    # Display the title
    print("\n========== CHATBOT AGENT ==========\n")

    # Display PEAS Representation
    print("PEAS Representation")
    print("-----------------------------")

    # Performance Measure (Goal of the agent)
    print("Performance Measure : Provide correct and helpful responses.")

    # Environment in which the chatbot works
    print("Environment         : User interacting through keyboard.")

    # Actuator used by the chatbot
    print("Actuators           : Display text response.")

    # Sensor used by the chatbot
    print("Sensors             : Read user's text input.")

    print("-----------------------------\n")

# Function to process user input and generate a response
def respond(self, message):

    # Convert the user's message into lowercase
    # so that comparison becomes case-insensitive
    message = message.lower()

    # Check whether the user says Hello
    if "hello" in message:
        return "Hello! How can I help you?"

    # Check whether the user asks the chatbot's name
    elif "name" in message:
        return "I am an Intelligent Chatbot."

    # Check whether the user asks about the course
    elif "course" in message:
        return "This is Artificial Intelligence and Machine Learning Lab."

    # Check whether the user asks for the current time
    elif "time" in message:

        # Import datetime module
        import datetime
```

```
            # Return the current system time
            return "Current Time : " + datetime.datetime.now().strftime("%H:%M:%S")

        # Check whether the user wants to exit
        elif "bye" in message:
            return "Thank you! Have a nice day."

        # Default response if no condition matches
        else:
            return "Sorry, I don't understand your question."


# ---------- Main Program Starts ----------

# Create an object of ChatbotAgent class
agent = ChatbotAgent()

# Display the working steps of the intelligent agent
print("Agent Behavior")
print("------------------------------")
print("1. Sense  -> Read user input")
print("2. Think  -> Process the input")
print("3. Act    -> Display response")
print("4. Repeat -> Continue until user types 'bye'")
print("------------------------------")

# Infinite loop to keep chatbot running
while True:

    # ---------- SENSOR ----------
    # Read input from the user
    user = input("\nYou : ")

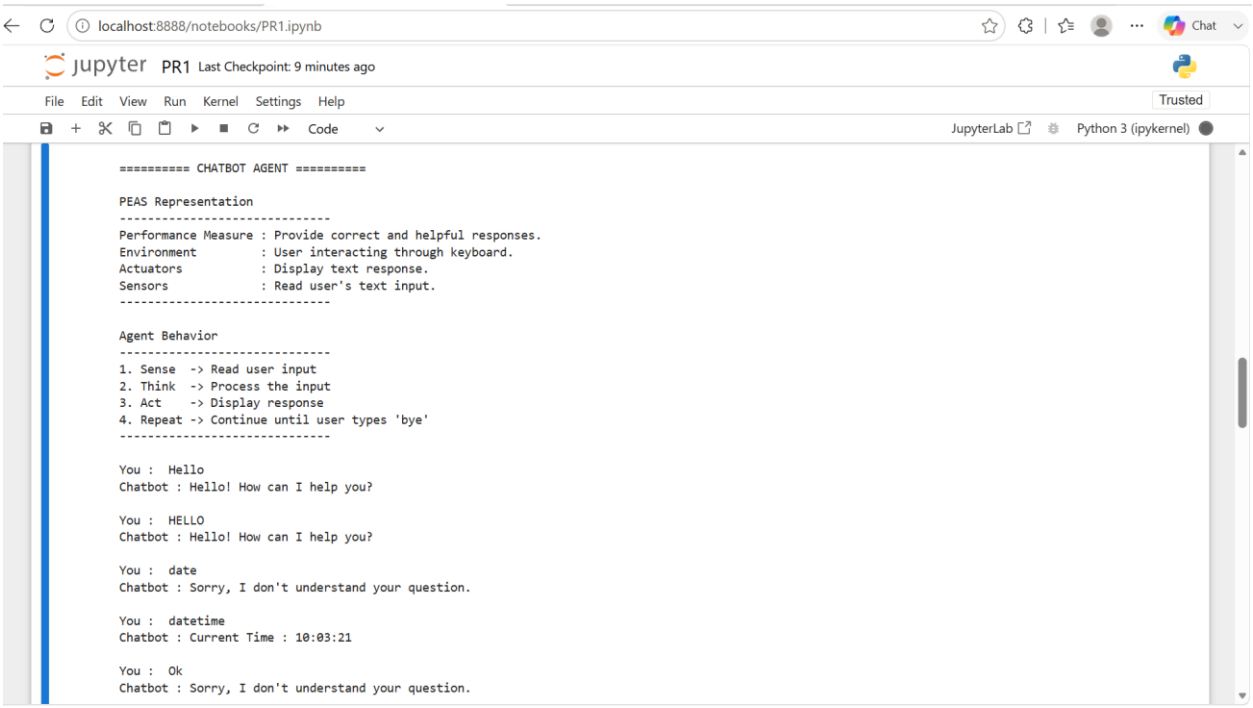
    # ---------- THINK ----------
    # Pass user input to the intelligent agent
    reply = agent.respond(user)

    # ---------- ACTUATOR ----------
    # Display chatbot response
    print("Chatbot :", reply)

    # Stop the chatbot if the user enters "bye"
    if user.lower() == "bye":
        break

# Program ends here
```

## OUTPUT




**Conclusion:** Understood and implemented intelligent chatbot agent.

# Assignment No: 02

## PR2: Uninformed Search Algorithms (CO1, CO2)

Implement Breadth First Search (BFS) and Depth First Search (DFS) algorithms for solving route-finding or maze navigation problems. Compare their performance based on path length, nodes explored and execution time

**Aim**

To implement Breadth First Search (BFS) and Depth First Search (DFS) algorithms for solving a maze navigation (route-finding) problem and compare their performance based on path length, nodes explored, and execution time.

**Theory**

**Breadth First Search (BFS)**

Breadth First Search (BFS) is an uninformed search algorithm that explores all neighboring nodes at the current depth before moving to the next level.

**Working**

- Starts from the source node.
- Visits all immediate neighbours first.
- Uses a Queue (FIFO).
- Guarantees the shortest path in an unweighted graph.

**Advantages**

- Finds the shortest path.
- Complete algorithm.
- Easy to implement.

**Disadvantages**

- Requires more memory.
- Slower for very large search spaces.

**Depth First Search (DFS)**

Depth First Search (DFS) explores one branch completely before backtracking to explore another branch.

**Working**

- Starts from the source node.
- Goes as deep as possible.
- Uses a Stack (LIFO) or recursion.
- Does not always find the shortest path.

**Advantages**

- Requires less memory.
- Simple recursive implementation.
- Faster in deep search spaces.

**Disadvantages**

- May not find the shortest path.
- Can get trapped in deep branches.

**Algorithm**
Algorithm for BFS
Step 1: Start.
Step 2: Create an empty queue.
Step 3: Insert the start node into the queue.
Step 4: Mark the start node as visited.
Step 5: While the queue is not empty:

- Remove the first node.
- If it is the goal, stop.
- Otherwise, visit all unvisited neighbouring nodes.
- Mark them visited.
- Add them to the queue.

Step 6: Display the shortest path.
Step 7: Stop.

**Algorithm for DFS**
**Step 1:** Start.
**Step 2:** Create an empty stack (or use recursion).
**Step 3:** Push the start node.
**Step 4:** Mark the node as visited.
**Step 5:** While the stack is not empty:

- Pop the top node.
- If it is the goal, stop.
- Otherwise, push all unvisited neighbouring nodes.

**Step 6:** Continue until the goal is found.

**Step 7:** Stop.

## Pseudocode

**BFS**

```
BEGIN

Create Queue
Mark Start as Visited
Enqueue(Start)

WHILE Queue is not empty

   Current = Dequeue()

   IF Current == Goal
      Display Path
      STOP
```

```
    FOR each Neighbor
        IF Neighbor not Visited
            Mark Visited
            Store Parent
            Enqueue(Neighbor)
END WHILE
END
```

**DFS**

```
 BEGIN
Create Stack
Push(Start)
Mark Start Visited
WHILE Stack is not empty
    Current = Pop()
    IF Current == Goal
        Display Path
        STOP
    FOR each Neighbor
        IF Neighbor not Visited
            Mark Visited
            Store Parent
            Push(Neighbor)
END WHILE
END
```

**Code**

```
# Import deque for implementing BFS queue
from collections import deque

# Import time module to measure execution time
import time
# --------------------------------
# Graph Representation
# --------------------------------
# Dictionary representing connections between cities/nodes
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': ['G'],
    'G': []
}

# -----------------------------------
# Breadth First Search (BFS)
# -----------------------------------
def bfs(graph, start, goal):

    # Queue stores current node and path
    queue = deque([(start, [start])])

    # Set to keep track of visited nodes
    visited = set()

    # Counter for explored nodes
    nodes_explored = 0

    # Continue until queue becomes empty
    while queue:

        # Remove first element from queue
        node, path = queue.popleft()

        # Ignore if node already visited
        if node in visited:
            continue

        # Mark node as visited
        visited.add(node)
```

```
        # Increase explored node count
        nodes_explored += 1

        # Goal found
        if node == goal:
            return path, nodes_explored

        # Visit all neighboring nodes
        for neighbor in graph[node]:
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))

    # Return None if no path exists
    return None, nodes_explored


# ------------------------------------
# Depth First Search (DFS)
# ------------------------------------
def dfs(graph, start, goal):

    # Stack stores current node and path
    stack = [(start, [start])]

    # Set for visited nodes
    visited = set()

    # Counter for explored nodes
    nodes_explored = 0

    # Continue until stack becomes empty
    while stack:

        # Remove last element from stack
        node, path = stack.pop()

        # Ignore visited nodes
        if node in visited:
            continue

        # Mark node as visited
        visited.add(node)

        # Increase explored node count
        nodes_explored += 1

        # Goal found
        if node == goal:
```

```
            return path, nodes_explored

        # Push neighbors into stack
        # Reverse order keeps traversal natural
        for neighbor in reversed(graph[node]):
            if neighbor not in visited:
                stack.append((neighbor, path + [neighbor]))

    # Return None if goal not found
    return None, nodes_explored


# ------------------------------------
# Main Program
# ------------------------------------

# Define starting node
start_node = 'A'

# Define goal node
goal_node = 'G'

# ---------------- BFS ----------------

# Record start time
start_time = time.perf_counter()

# Run BFS
bfs_path, bfs_nodes = bfs(graph, start_node, goal_node)

# Record end time
end_time = time.perf_counter()

# Calculate execution time
bfs_time = end_time - start_time

# ---------------- DFS ----------------

# Record start time
start_time = time.perf_counter()

# Run DFS
dfs_path, dfs_nodes = dfs(graph, start_node, goal_node)

# Record end time
end_time = time.perf_counter()

# Calculate execution time
dfs_time = end_time - start_time
```

```
# ------------------------------------
# Display Results
# ------------------------------------

print("\n===== Breadth First Search (BFS) =====")
print("Path :", bfs_path)
print("Path Length :", len(bfs_path)-1)
print("Nodes Explored :", bfs_nodes)
print("Execution Time :", bfs_time, "seconds")

print("\n===== Depth First Search (DFS) =====")
print("Path :", dfs_path)
print("Path Length :", len(dfs_path)-1)
print("Nodes Explored :", dfs_nodes)
print("Execution Time :", dfs_time, "seconds")

# ------------------------------------
# Performance Comparison
# ------------------------------------

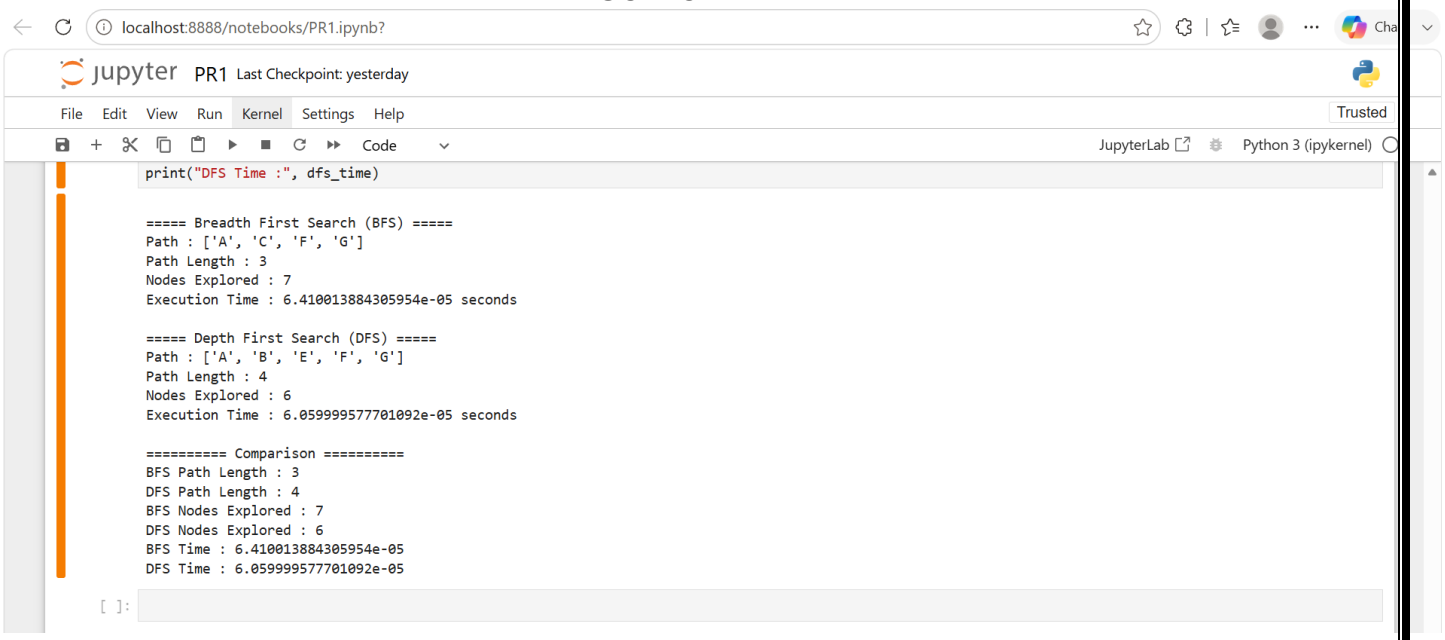
print("\n========== Comparison ==========")

print("BFS Path Length :", len(bfs_path)-1)
print("DFS Path Length :", len(dfs_path)-1)

print("BFS Nodes Explored :", bfs_nodes)
print("DFS Nodes Explored :", dfs_nodes)

print("BFS Time :", bfs_time)
print("DFS Time :", dfs_time)
```

## OUTPUT




**Conclusion:** Understood and implemented Breadth First Search (BFS) and Depth First Search (DFS) were successfully implemented for solving a maze navigation (route-finding) problem.

# Assignment No: 03

**PR3: Knowledge Representation and Expert System (CO1, CO2)**
Develop a rule-based expert system using propositional logic and implement Forward Chaining and Backward Chaining mechanisms for intelligent decision making.

**Aim**
To develop a **rule-based expert system** using **propositional logic** and implement **Forward Chaining** and **Backward Chaining** inference mechanisms for intelligent decision making.

**Theory**
**Expert System**
An **Expert System** is an Artificial Intelligence (AI) program that mimics the decision-making ability of a human expert. It uses a collection of **facts** and **IF–THEN rules** stored in a **knowledge base** to solve problems and provide recommendations.
An expert system mainly consists of:

- **Knowledge Base:** Stores facts and rules.
- **Inference Engine:** Applies logical reasoning to derive conclusions.
- **User Interface:** Allows users to interact with the system.

**Propositional Logic**
**Propositional Logic** is a branch of logic in which statements are represented as propositions that can be either **True** or **False**. It forms the basis for rule-based reasoning in expert systems.
Example:

- P = Patient has fever
- Q = Patient has cough
- R = Patient has flu

Rule:
**IF P AND Q THEN R**
Meaning:
If a patient has both fever and cough, then the patient may have flu.

**Rule-Based Expert System**
A rule-based expert system stores knowledge in the form of **IF–THEN rules**.
Example Rules:

- IF Fever AND Cough THEN Flu
- IF Flu THEN Take Medicine
- IF High Temperature THEN Fever

The inference engine uses these rules to derive new facts.

**Forward Chaining**
Forward Chaining is a **data-driven** reasoning technique.
It starts with known facts and repeatedly applies rules to infer new facts until the goal is reached.
**Working**

1. Start with known facts.
2. Check all rules.
3. Fire rules whose conditions are satisfied.

4. Add newly inferred facts.
5. Repeat until no new facts are generated or the goal is achieved.

**Advantages**

- Finds all possible conclusions.
- Suitable when many facts are available.
- Automatically derives new knowledge.

**Disadvantages**

- May evaluate unnecessary rules.
- Can be slower for large knowledge bases.

**Backward Chaining**

Backward Chaining is a **goal-driven** reasoning technique.
It starts with a goal and works backward to determine whether the goal can be proved using known facts and rules.

**Working**

1. Select the goal.
2. Find a rule that concludes the goal.
3. Check whether the rule's conditions are true.
4. If not, prove those conditions recursively.
5. Continue until the goal is proved or fails.

**Advantages**

- Efficient for answering specific queries.
- Checks only relevant rules.
- Reduces unnecessary computation.

**Disadvantages**

- Not suitable when all possible conclusions are required.
- Depends on a predefined goal.

**Example Knowledge Base**

**Facts**

- Fever = True
- Cough = True

**Rules**

| Rule No. | Rule |
|---|---|
| R1 | IF Fever AND Cough THEN Flu |
| R2 | IF Flu THEN Take_Medicine |
| R3 | IF High_Temperature THEN Fever |

**Algorithm**

**Algorithm for Forward Chaining**

**Step 1:** Start.
**Step 2:** Read the initial facts.
**Step 3:** Store all IF–THEN rules.
**Step 4:** Compare facts with rule conditions.
**Step 5:** If all conditions of a rule are satisfied, infer the conclusion.
**Step 6:** Add the new conclusion to the fact base.
**Step 7:** Repeat Steps 4–6 until no new facts are generated.
**Step 8:** Display the inferred conclusions.

**Step 9:** Stop.

**Algorithm for Backward Chaining**
**Step 1:** Start.
**Step 2:** Select the goal.
**Step 3:** Search for a rule whose conclusion matches the goal.
**Step 4:** Check whether the rule's conditions are already known.
**Step 5:** If not, recursively prove each condition.
**Step 6:** If all conditions are true, conclude that the goal is true.
**Step 7:** Otherwise, conclude that the goal cannot be proved.
**Step 8:** Stop.

**Pseudocode**
**Forward Chaining**

```
BEGIN

Input Initial Facts

Input Rules

REPEAT

   FOR each Rule

      IF Rule Conditions are True

         AND Conclusion not already known

            Add Conclusion to Facts

UNTIL no new facts generated

Display Final Facts

END
```

**Backward Chaining**

```
BEGIN

Input Goal

IF Goal already exists in Facts

   RETURN TRUE

Find Rule whose Conclusion = Goal
```

FOR each Condition

Recursively Prove Condition

IF all Conditions are TRUE

RETURN TRUE

ELSE

RETURN FALSE

END

**code**

```
 # ------------------------------------------------------------
# Rule-Based Expert System using Propositional Logic
# Implements:
# 1. Forward Chaining
# 2. Backward Chaining
# Knowledge Base (Rules)
# ----------------------------
# Each rule consists of:
# IF (conditions) THEN (conclusion)

rules = [
    (["fever", "cough"], "flu"),                # Rule 1
    (["flu"], "visit_doctor"),                  # Rule 2
    (["headache", "fever"], "viral_infection"),   # Rule 3
    (["viral_infection"], "take_rest"),          # Rule 4
    (["take_rest"], "recover"),                 # Rule 5
]

# ------------------------------------------------------------
# Forward Chaining Function
# ------------------------------------------------------------
def forward_chaining(facts, rules):
    # Convert initial facts into a set to avoid duplicates
    inferred = set(facts)

    # Variable to check whether new facts are added
    changed = True

    # Repeat until no new fact is inferred
    while changed:

        # Assume nothing new is inferred
        changed = False
```

```
        # Check every rule
        for conditions, conclusion in rules:

            # Verify whether all conditions are true
            if all(condition in inferred for condition in conditions):

                # If conclusion is not already known
                if conclusion not in inferred:

                    # Add new conclusion
                    inferred.add(conclusion)

                    # Print newly inferred fact
                    print("Forward Chaining Inferred:", conclusion)

                    # Continue another iteration
                    changed = True

    # Return all inferred facts
    return inferred


# ------------------------------------------------------------
# Backward Chaining Function
# ------------------------------------------------------------
def backward_chaining(goal, facts, rules):
    # If goal already exists in facts, return True
    if goal in facts:
        return True

    # Check every rule
    for conditions, conclusion in rules:

        # Find rule that produces the goal
        if conclusion == goal:

            # Assume all conditions are true
            result = True

            # Check every condition recursively
            for condition in conditions:

                # If any condition cannot be proved
                if not backward_chaining(condition, facts, rules):
                    result = False
                    break

            # If every condition is proved
            if result:
```

```
            return True

    # Goal cannot be proved
    return False
# ------------------------------------------------------------
# Main Program
# ------------------------------------------------------------

# Initial known facts
facts = {"fever", "cough"}

# Display initial facts
print("Initial Facts:", facts)

print("\n----- Forward Chaining -----")

# Perform Forward Chaining
all_facts = forward_chaining(facts, rules)

# Display all inferred facts
print("\nFinal Facts after Forward Chaining:")
print(all_facts)

print("\n----- Backward Chaining -----")

# Goal to prove
goal = "visit_doctor"

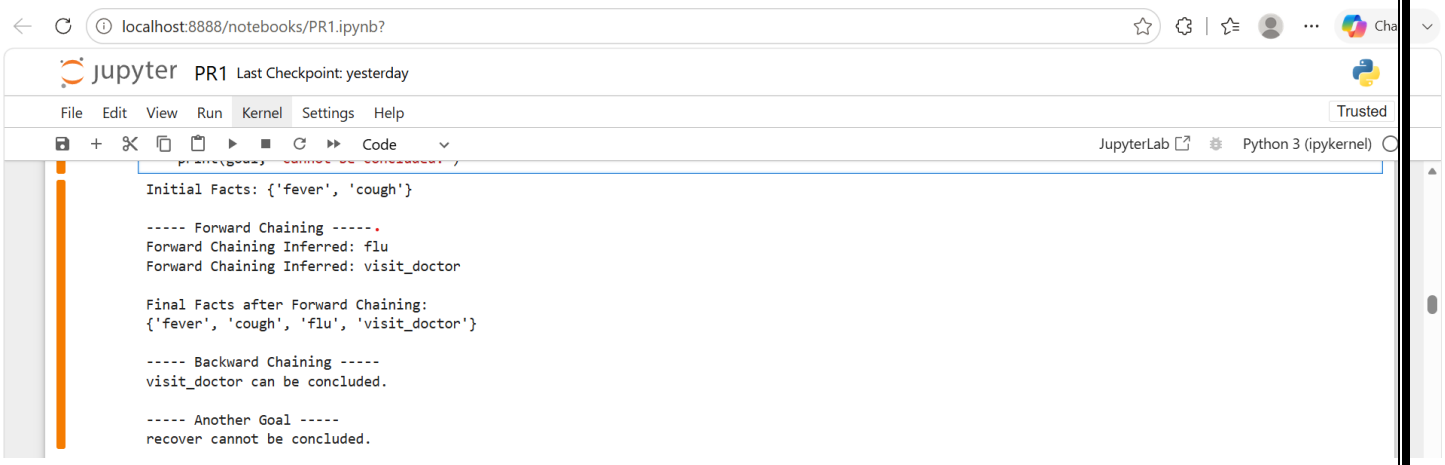
# Check whether goal can be proved
if backward_chaining(goal, facts, rules):
    print(goal, "can be concluded.")
else:
    print(goal, "cannot be concluded.")

print("\n----- Another Goal -----")

# Check another goal
goal = "recover"

# Verify second goal
if backward_chaining(goal, facts, rules):
    print(goal, "can be concluded.")
else:
    print(goal, "cannot be concluded.")
```

## OUTPUT




**Conclusion:** Understood and implemented A rule-based expert system was successfully developed using propositional logic.

# Assignment No: 04

## PR4: Data Preprocessing and Exploratory Data Analysis (CO2)

Download a standard dataset (Heart Disease, Diabetes or Titanic dataset) and perform:

- Data cleaning
- Missing value handling
- Feature scaling
- Encoding techniques
- Statistical analysis
- Data visualization using histograms and boxplots

Split the dataset into training and testing sets.

**Aim:**

To download a standard dataset (Heart Disease, Diabetes, or Titanic dataset) and perform Data Cleaning, Missing Value Handling, Feature Scaling, Encoding Techniques, Statistical Analysis, and Data Visualization using Histograms and Boxplots.

**Theory**

**Data Preprocessing**

Data preprocessing is the process of converting raw data into a clean and understandable format before applying machine learning algorithms. It improves data quality and model performance.

**The main preprocessing steps include:**

- Data Cleaning
- Missing Value Handling
- Feature Scaling
- Encoding Categorical Variables

**Exploratory Data Analysis (EDA)**

Exploratory Data Analysis (EDA) is the process of understanding the dataset by analyzing its structure, distribution, relationships, and identifying patterns using statistical methods and visualizations.

**EDA helps to:**

- Understand data distribution
- Detect outliers
- Find relationships between variables
- Prepare data for machine learning

**Dataset**

**Dataset Used: Titanic Dataset**

The dataset contains information about passengers such as:

- PassengerId
- Pclass
- Name
- Sex
- Age

- Fare
- Embarked
- Survival Status

**Data Preprocessing Steps**

**1. Data Cleaning**

- Remove duplicate records.
- Correct inconsistent values.
- Remove unnecessary columns (if required).

**2. Missing Value Handling**

- Fill numerical missing values using the mean or median.
- Fill categorical missing values using the mode.

**3. Feature Scaling**

- Normalize or standardize numerical features such as Age and Fare using StandardScaler.

**4. Encoding Techniques**

- Convert categorical variables (Sex, Embarked) into numerical values using Label Encoding or One-Hot Encoding.

**5. Statistical Analysis**

- **Calculate:**
  - Mean
  - Median
  - Standard Deviation
  - Minimum
  - Maximum
  - Quartiles

**6. Data Visualization**

- Histogram
- Boxplot

**Algorithm**

Step 1:
Import the required Python libraries.
Step 2:
Load the Titanic dataset.
Step 3:
Display the dataset information.
Step 4:
Check for missing values.
Step 5:
Handle missing values using mean and mode.
Step 6:
Remove duplicate records.
Step 7:
Encode categorical variables.
Step 8:
Apply feature scaling.
Step 9:
Perform statistical analysis.
Step 10:

Visualize the dataset using histograms and boxplots.
Step 11:
Display the processed dataset.
Step 12:
Stop.

## Pseudocode

BEGIN

Load Dataset

Display Dataset Information

Check Missing Values

Fill Missing Numerical Values

Fill Missing Categorical Values

Remove Duplicate Records

Encode Categorical Features

Scale Numerical Features

Calculate Statistical Summary

Plot Histogram

Plot Boxplot

Display Clean Dataset

END

## Code

```
#-------------------------------------------------------
# Data Preprocessing on Titanic Dataset
# -------------------------------------------------------

# Import required libraries
import pandas as pd                    # For data manipulation
import numpy as np                     # For numerical operations
import matplotlib.pyplot as plt        # For plotting graphs
from sklearn.model_selection import train_test_split   # To split dataset
from sklearn.preprocessing import StandardScaler       # For feature scaling
from sklearn.preprocessing import LabelEncoder         # For encoding categorical data
```

```
# ---------------------------------------------------------
# Load Dataset
# ---------------------------------------------------------

# Load Titanic dataset from a CSV file
# Replace "Titanic.csv" with your dataset path if necessary
df = pd.read_csv("train.csv")

# Display first 5 rows of dataset
print("First Five Records")
print(df.head())

# ---------------------------------------------------------
# Basic Information
# ---------------------------------------------------------

# Display dataset information
print("\nDataset Information")
print(df.info())

# Display statistical summary
print("\nStatistical Summary")
print(df.describe())

# ---------------------------------------------------------
# Data Cleaning
# ---------------------------------------------------------

# Check duplicate records
print("\nDuplicate Records:", df.duplicated().sum())

# Remove duplicate records if present
df.drop_duplicates(inplace=True)

# ---------------------------------------------------------
# Missing Value Handling
# ---------------------------------------------------------

# Display missing values in each column
print("\nMissing Values")
print(df.isnull().sum())

# Fill missing Age values using mean
df["Age"].fillna(df["Age"].mean(), inplace=True)

# Fill missing Embarked values using most frequent value
df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
```

```
# Fill missing Fare values using mean
df["Fare"].fillna(df["Fare"].mean(), inplace=True)

# Drop Cabin column because it contains many missing values
if "Cabin" in df.columns:
    df.drop("Cabin", axis=1, inplace=True)

# ------------------------------------------------------
# Encoding Categorical Variables
# ------------------------------------------------------

# Create Label Encoder object
encoder = LabelEncoder()

# Convert Sex column into numerical values
# male = 1, female = 0 (or vice versa)
df["Sex"] = encoder.fit_transform(df["Sex"])

# Convert Embarked column into numerical values
df["Embarked"] = encoder.fit_transform(df["Embarked"])

# ------------------------------------------------------
# Feature Scaling
# ------------------------------------------------------

# Create StandardScaler object
scaler = StandardScaler()

# Scale numerical features
df[["Age", "Fare"]] = scaler.fit_transform(df[["Age", "Fare"]])

# ------------------------------------------------------
# Statistical Analysis
# ------------------------------------------------------

print("\nMean of Numerical Columns")
print(df.mean(numeric_only=True))

print("\nMedian of Numerical Columns")
print(df.median(numeric_only=True))

print("\nStandard Deviation")
print(df.std(numeric_only=True))

print("\nCorrelation Matrix")
print(df.corr(numeric_only=True))

# ------------------------------------------------------
# Data Visualization
```

```
# ------------------------------------------------------

# Histogram of Age
plt.figure(figsize=(2,2))
plt.hist(df["Age"], bins=20)
plt.title("Histogram of Age")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()

# Histogram of Fare
plt.figure(figsize=(2,2))
plt.hist(df["Fare"], bins=20)
plt.title("Histogram of Fare")
plt.xlabel("Fare")
plt.ylabel("Frequency")
plt.show()

# Boxplot of Age
plt.figure(figsize=(2,2))
plt.boxplot(df["Age"])
plt.title("Boxplot of Age")
plt.ylabel("Age")
plt.show()

# Boxplot of Fare
plt.figure(figsize=(2,2))
plt.boxplot(df["Fare"])
plt.title("Boxplot of Fare")
plt.ylabel("Fare")
plt.show()

# ------------------------------------------------------
# Split Dataset into Features and Target
# ------------------------------------------------------

# Features (Input)
X = df.drop("Survived", axis=1)

# Target (Output)
y = df["Survived"]

# ------------------------------------------------------
# Remove Non-Numeric Columns (if present)
# ------------------------------------------------------

# Drop Name, Ticket and PassengerId if they exist
columns_to_remove = ["Name", "Ticket", "PassengerId"]
```

```
for column in columns_to_remove:
    if column in X.columns:
        X = X.drop(column, axis=1)

# ---------------------------------------------------------
# Split Dataset
# ---------------------------------------------------------

# Split dataset into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split( X, # Features  y, # Target
    test_size=0.20,         # 20% test data
    random_state=42         # Same split every execution
)

# ---------------------------------------------------------
# Display Dataset Sizes
# ---------------------------------------------------------

print("\nTraining Feature Shape :", X_train.shape)
print("Testing Feature Shape  :", X_test.shape)

print("Training Target Shape  :", y_train.shape)
print("Testing Target Shape   :", y_test.shape)

print("\nData Preprocessing Completed Successfully!")
```

**OUTPUT**

```
First Five Records
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
```

```
   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

```
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
```

```
Statistical Summary
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000
```

```
             Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
```

```
Duplicate Records: 0

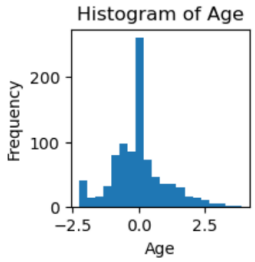
Missing Values
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

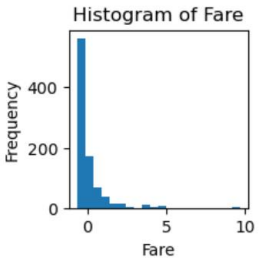
```
Mean of Numerical Columns
PassengerId    4.460000e+02
Survived       3.838384e-01
Pclass         2.308642e+00
Sex            6.475870e-01
Age            2.232906e-16
SibSp          5.230079e-01
Parch          3.815937e-01
Fare           3.987333e-18
Embarked       1.536476e+00
dtype: float64
```

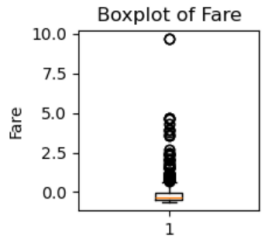
```
Median of Numerical Columns
PassengerId    446.000000
Survived         0.000000
Pclass           3.000000
Sex              1.000000
Age              0.000000
SibSp            0.000000
Parch            0.000000
Fare            -0.357391
Embarked         2.000000
dtype: float64
```

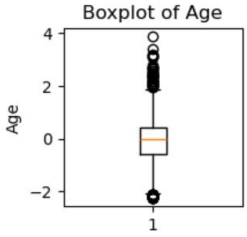
```
Standard Deviation
PassengerId    257.353842
Survived         0.486592
Pclass           0.836071
Sex              0.477990
Age              1.000562
SibSp            1.102743
Parch            0.806057
Fare             1.000562
Embarked         0.791503
dtype: float64
```

```
Correlation Matrix
             PassengerId  Survived    Pclass       Sex       Age     SibSp  \
PassengerId     1.000000 -0.005007 -0.035144  0.042939  0.033207 -0.057527
Survived       -0.005007  1.000000 -0.338481 -0.543351 -0.069809 -0.035322
Pclass         -0.035144 -0.338481  1.000000  0.131900 -0.331339  0.083081
Sex             0.042939 -0.543351  0.131900  1.000000  0.084153 -0.114631
Age             0.033207 -0.069809 -0.331339  0.084153  1.000000 -0.232625
SibSp          -0.057527 -0.035322  0.083081 -0.114631 -0.232625  1.000000
Parch          -0.001652  0.081629  0.018443 -0.245489 -0.179191  0.414838
Fare            0.012658  0.257307 -0.549500 -0.182333  0.091566  0.159651
Embarked        0.013128 -0.167675  0.162098  0.108262 -0.026749  0.068230
```




```
Training Feature Shape : (712, 7)
Testing Feature Shape  : (179, 7)
Training Target Shape  : (712,)
Testing Target Shape   : (179,)

Data Preprocessing Completed Successfully!
```

**Conclusion:** Understood and implemented The Titanic dataset was successfully preprocessed by performing data cleaning, missing value handling, feature scaling, and encoding of categorical variable

# Assignment No: 05

## PR5: Regression Techniques (CO2, CO3)

Implement Linear Regression and Polynomial Regression on a suitable dataset such as California Housing or House Price Prediction dataset.
Evaluate model performance using:

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- $R^2$ Score

Compare the results obtained by both techniques.

**Aim:**

To implement **Linear Regression** and **Polynomial Regression** on a suitable dataset (California Housing or House Price Prediction dataset) and evaluate their performance using **Mean Squared Error (MSE), Mean Absolute Error (MAE), and $R^2$ Score**. Finally, compare the results obtained by both techniques.

**Theory**

**Linear Regression**

Linear Regression is a supervised machine learning algorithm used to predict continuous numerical values. It models the relationship between independent variables (features) and the dependent variable (target) using a straight line.

**Equation**

$$Y = b_0 + b_1 X$$

Where:

- **Y** = Predicted value
- **X** = Input feature
- **$b_0$** = Intercept
- **$b_1$** = Slope (coefficient)

**Advantages**

- Simple and easy to understand.
- Fast training.
- Works well for linear relationships.

**Disadvantages**

- Cannot model nonlinear relationships.
- Sensitive to outliers.

**Polynomial Regression**

Polynomial Regression is an extension of Linear Regression that models nonlinear relationships by introducing polynomial terms.

**Equation**

$$Y = b_0 + b_1 X + b_2 X^2 + b_3 X^3 + \cdots + b_n X^n$$

It converts linear features into polynomial features before applying linear regression.

**Advantages**

- Models nonlinear relationships.
- More flexible than linear regression.
- Higher prediction accuracy for curved data.

**Disadvantages**

- More computationally expensive.
- Can overfit if the polynomial degree is too high.

**Performance Evaluation Metrics**

**1. Mean Squared Error (MSE)**

Measures the average squared difference between actual and predicted values.

$$MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

**Lower MSE indicates better performance.**

**2. Mean Absolute Error (MAE)**

Measures the average absolute difference between actual and predicted values.

$$MAE = \frac{1}{n}\sum | y_i - \hat{y}_i |$$

**Lower MAE indicates better performance.**

**3. $R^2$ Score (Coefficient of Determination)**

Measures how well the model explains the variation in the target variable.

$$R^2 = 1 - \frac{SS_{Residual}}{SS_{Total}}$$

**Higher $R^2$ (closer to 1) indicates better performance.**

**Algorithm**

**Algorithm for Linear Regression**

**Step 1:** Start.
**Step 2:** Import the required libraries.
**Step 3:** Load the California Housing or House Price Prediction dataset.
**Step 4:** Split the dataset into training and testing sets.
**Step 5:** Train the Linear Regression model.
**Step 6:** Predict house prices using the test data.
**Step 7:** Calculate MSE, MAE, and $R^2$ Score.
**Step 8:** Display the results.
**Step 9:** Stop.

**Algorithm for Polynomial Regression**

**Step 1:** Start.
**Step 2:** Load the dataset.
**Step 3:** Convert features into polynomial features.
**Step 4:** Train the Polynomial Regression model.
**Step 5:** Predict house prices.
**Step 6:** Calculate MSE, MAE, and $R^2$ Score.
**Step 7:** Compare the performance with Linear Regression.
**Step 8:** Stop.

**Pseudocode**

**Linear Regression**

BEGIN

Load Dataset

Split Dataset into Train and Test

Train Linear Regression Model

Predict Test Data

Calculate MSE

Calculate MAE

Calculate $R^2$ Score

Display Results

END

**Polynomial Regression**

BEGIN

Load Dataset

Generate Polynomial Features

Split Dataset

Train Polynomial Regression Model

Predict Test Data

Calculate MSE

Calculate MAE

Calculate $R^2$ Score

Compare with Linear Regression

END

## Code

```
# ------------------------------------------------------------
# Linear Regression and Polynomial Regression
# California Housing Dataset
# Evaluation using MSE, MAE and R² Score
# ------------------------------------------------------------

# Import NumPy library for numerical operations
import numpy as np

# Import pandas library for data manipulation
import pandas as pd

# Import matplotlib for plotting graphs
import matplotlib.pyplot as plt

# Import California Housing dataset
from sklearn.datasets import fetch_california_housing

# Import function to split dataset into training and testing sets
from sklearn.model_selection import train_test_split

# Import Linear Regression model
from sklearn.linear_model import LinearRegression

# Import PolynomialFeatures for Polynomial Regression
from sklearn.preprocessing import PolynomialFeatures

# Import performance evaluation metrics
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import r2_score

# ------------------------------------------------------------
# Load Dataset
# ------------------------------------------------------------

# Load California Housing dataset
housing = fetch_california_housing()

# Convert dataset into a pandas DataFrame
df = pd.DataFrame(housing.data, columns=housing.feature_names)

# Add target (House Price) column to DataFrame
df["HousePrice"] = housing.target

# Display first five records
```

```
print("First Five Records")
print(df.head())

# ------------------------------------------------------------
# Separate Features and Target
# ------------------------------------------------------------

# Store input features in X
X = df.drop("HousePrice", axis=1)

# Store target variable in y
y = df["HousePrice"]

# ------------------------------------------------------------
# Split Dataset
# ------------------------------------------------------------

# Split dataset into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X,                  # Input features
    y,                  # Target variable
    test_size=0.20,         # 20% Testing Data
    random_state=42         # Fixed random state
)

# ------------------------------------------------------------
# Linear Regression
# ------------------------------------------------------------

# Create Linear Regression model
linear_model = LinearRegression()

# Train Linear Regression model
linear_model.fit(X_train, y_train)

# Predict testing data
linear_predictions = linear_model.predict(X_test)

# ------------------------------------------------------------
# Evaluate Linear Regression
# ------------------------------------------------------------

# Calculate Mean Squared Error
linear_mse = mean_squared_error(y_test, linear_predictions)

# Calculate Mean Absolute Error
linear_mae = mean_absolute_error(y_test, linear_predictions)

# Calculate R² Score
```

```
linear_r2 = r2_score(y_test, linear_predictions)

# Print Linear Regression Results
print("\n----- Linear Regression Results -----")

print("Mean Squared Error :", linear_mse)

print("Mean Absolute Error :", linear_mae)

print("R2 Score :", linear_r2)

# ------------------------------------------------------------
# Polynomial Regression
# ------------------------------------------------------------

# Create PolynomialFeatures object with degree 2
poly = PolynomialFeatures(degree=2)

# Transform training data into polynomial features
X_train_poly = poly.fit_transform(X_train)

# Transform testing data into polynomial features
X_test_poly = poly.transform(X_test)

# Create Linear Regression model
poly_model = LinearRegression()

# Train Polynomial Regression model
poly_model.fit(X_train_poly, y_train)

# Predict testing data
poly_predictions = poly_model.predict(X_test_poly)

# ------------------------------------------------------------
# Evaluate Polynomial Regression
# ------------------------------------------------------------

# Calculate Mean Squared Error
poly_mse = mean_squared_error(y_test, poly_predictions)

# Calculate Mean Absolute Error
poly_mae = mean_absolute_error(y_test, poly_predictions)

# Calculate R² Score
poly_r2 = r2_score(y_test, poly_predictions)

# Print Polynomial Regression Results
print("\n----- Polynomial Regression Results -----")
```

```
print("Mean Squared Error :", poly_mse)

print("Mean Absolute Error :", poly_mae)

print("R2 Score :", poly_r2)

# ------------------------------------------------------------
# Compare Both Models
# ------------------------------------------------------------

# Create comparison table
comparison = pd.DataFrame({
    "Metric": ["Mean Squared Error", "Mean Absolute Error", "R2 Score"],
    "Linear Regression": [linear_mse, linear_mae, linear_r2],
    "Polynomial Regression": [poly_mse, poly_mae, poly_r2]
})

# Display comparison table
print("\nComparison of Models")
print(comparison)

# ------------------------------------------------------------
# Visualization
# ------------------------------------------------------------

# Create a figure
plt.figure(figsize=(8,6))

# Scatter plot of actual house prices
plt.scatter(
    y_test,
    linear_predictions,
    color="blue",
    label="Linear Regression"
)

# Scatter plot of polynomial predictions
plt.scatter(
    y_test,
    poly_predictions,
    color="red",
    alpha=0.5,
    label="Polynomial Regression"
)

# Label X-axis
plt.xlabel("Actual House Prices")

# Label Y-axis
```

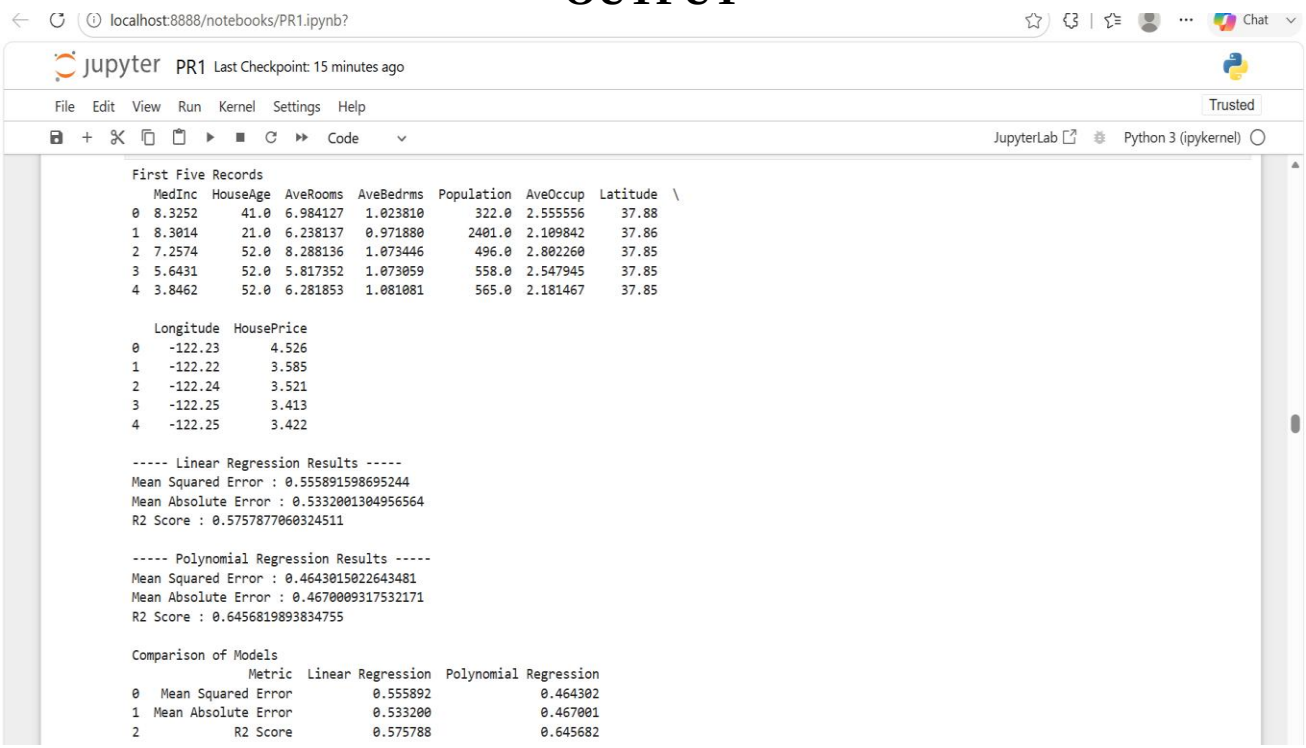
```
plt.ylabel("Predicted House Prices")

# Add graph title
plt.title("Actual vs Predicted House Prices")

# Display legend
plt.legend()

# Display graph
plt.show()
```

# OUTPUT



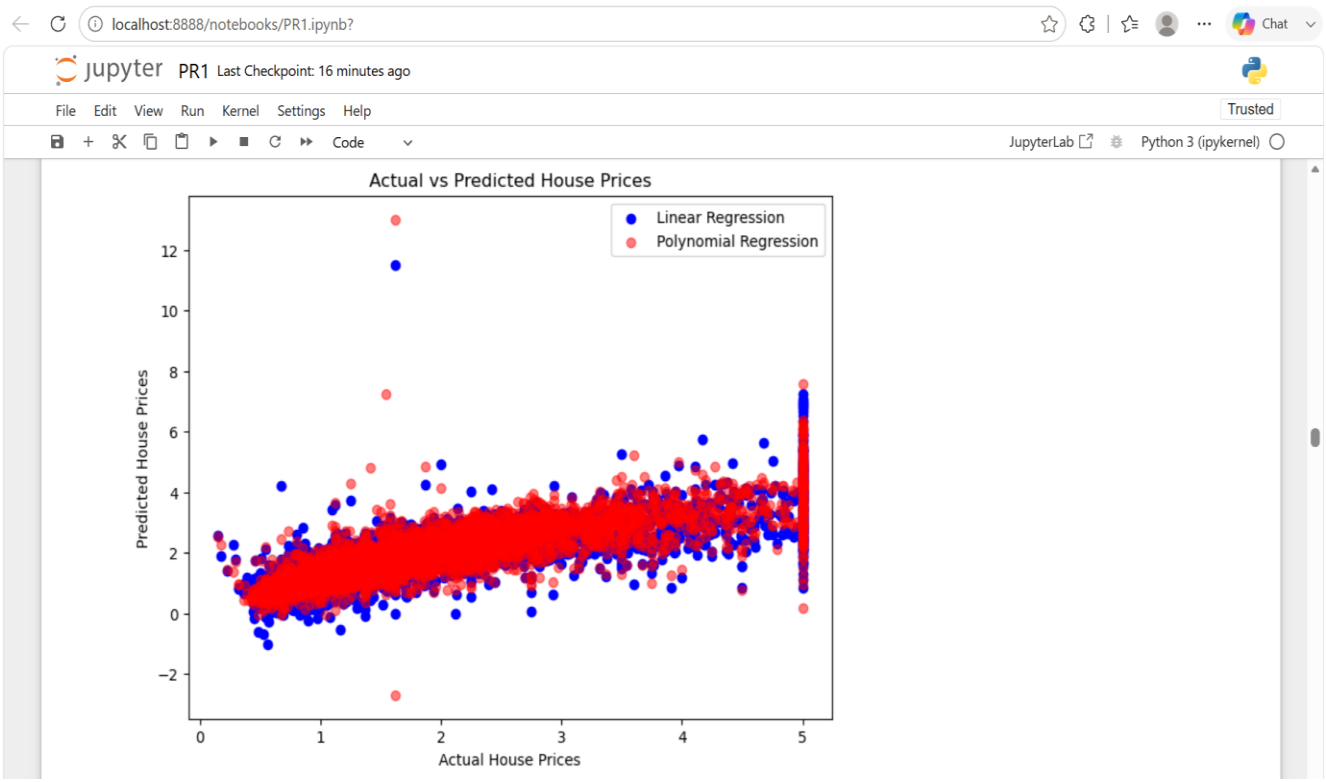

```
First Five Records
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  HousePrice
0    -122.23       4.526
1    -122.22       3.585
2    -122.24       3.521
3    -122.25       3.413
4    -122.25       3.422

----- Linear Regression Results -----
Mean Squared Error : 0.555891598695244
Mean Absolute Error : 0.5332001304956564
R2 Score : 0.5757877060324511

----- Polynomial Regression Results -----
Mean Squared Error : 0.4643015022643481
Mean Absolute Error : 0.4670009317532171
R2 Score : 0.6456819893834755

Comparison of Models
               Metric  Linear Regression  Polynomial Regression
0   Mean Squared Error           0.555892               0.464302
1  Mean Absolute Error           0.533200               0.467001
2            R2 Score           0.575788               0.645682
```




**Conclusion:** Linear Regression and Polynomial Regression were successfully implemented on a **House Price** Prediction/California Housing dataset.

# Assignment No: 06

## PR6: Classification using Logistic Regression and K-Nearest Neighbors (CO2, CO3)

### 1. Aim

To perform data preprocessing and implement Logistic Regression and K-Nearest Neighbors (KNN) classifiers on the Social Network Ads dataset, evaluate them using Accuracy, Precision, Recall, F1-Score and the Confusion Matrix, and analyze and compare the results.

### 2. Software and Tools Required

- Python 3 with Jupyter Notebook / Google Colab
- Libraries: NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn
- Dataset: Social_Network_Ads_Small.csv (30 records)

### 3. Theory

#### 3.1 Classification

Classification is a supervised learning task in which a model learns from labelled examples to assign new inputs to one of a set of predefined classes. In this practical the problem is binary: predict whether a social-network user purchases a product (1) or not (0) after seeing an advertisement, using Age and Estimated Salary as inputs.

#### 3.2 Logistic Regression

Logistic Regression is a linear, parametric classifier. It computes a weighted sum of the input features and passes it through the sigmoid function to obtain the probability of the positive class:

$$P(y = 1 \mid x) = 1 / (1 + e^{-(w1 \cdot x1 + w2 \cdot x2 + b)})$$

An instance is assigned to class 1 when this probability is at least 0.5. The decision boundary is a straight line in two dimensions. The model is fast and interpretable (the coefficients show how each feature pushes the prediction), but it can under-fit when the true class boundary is curved.

#### 3.3 K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based classifier. It stores the training data and, to classify a new point, finds the k closest training points (by Euclidean distance) and assigns the majority class among them. Its decision boundary is flexible and can follow local clusters. Its performance depends strongly on the value of k and on feature scaling, because it relies on distances. Odd values of k avoid tied votes in a binary problem.

#### 3.4 Feature Scaling

Age (about 18 to 60) and Estimated Salary (about 20,000 to 150,000) differ greatly in range. StandardScaler transforms each feature to zero mean and unit variance, z = (x - mean) / std, so that

Salary does not dominate Age only because of its larger numbers. The scaler is fitted on the training data only and then applied to the test data, to avoid data leakage.

### 3.5 Evaluation Metrics

For a binary problem, the confusion matrix counts True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN).

| Metric | Formula | Meaning |
|---|---|---|
| Accuracy | (TP + TN) / (TP + TN + FP + FN) | Fraction of all predictions that are correct |
| Precision | TP / (TP + FP) | Of predicted buyers, how many truly bought |
| Recall | TP / (TP + FN) | Of actual buyers, how many were found |
| F1-Score | 2 · P · R / (P + R) | Harmonic mean of precision and recall |

## 4. Dataset Description

The Social Network Ads dataset (small subset) has 30 records and four columns. Age and EstimatedSalary are used as input features, and Purchased is the target. Gender is excluded to keep the problem two-dimensional. There are no missing values and no duplicate rows.

| Column | Type | Description |
|---|---|---|
| Gender | Categorical | Male / Female (not used in this practical) |
| Age | Integer | Age of the user in years |
| EstimatedSalary | Integer | Estimated annual salary of the user |
| Purchased | Binary (0/1) | Target: 1 = purchased after the ad, 0 = did not |

## 5. Algorithm / Procedure

1. Import the required libraries and fix the random seed.
2. Load the dataset and inspect its shape, data types and summary statistics.
3. Check for missing values and duplicate rows; select the Age, EstimatedSalary and Purchased columns.
4. Check the class balance and visualize Age against Estimated Salary.
5. Split the data into 75% training and 25% testing sets (stratified on the target).
6. Standardize the features using StandardScaler (fit on train, transform both).
7. Train a Logistic Regression model and predict on the test set.
8. Evaluate KNN for k = 1 to 11, choose the best k, train the final KNN model and predict.

9. Compute Accuracy, Precision, Recall, F1-Score, confusion matrices and classification reports for both models.
10. Compare the models and write the result and conclusion.

### 6. Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report, ConfusionMatrixDisplay)

RANDOM_STATE = 42   # fixed seed for reproducible results
```

```
# Load data and inspect
df = pd.read_csv('Social_Network_Ads_Small.csv')
print("Shape of dataset:", df.shape)
df.info()
df.describe()

# Preprocessing: missing values, duplicates, column selection
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
df = df[['Age', 'EstimatedSalary', 'Purchased']]

# Class balance
print(df['Purchased'].value_counts())
```

```
# Feature-target split and train-test split (75:25, stratified)
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=RANDOM_STATE, stratify=y)

# Feature scaling (fit on training data only)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Logistic Regression
log_reg = LogisticRegression(random_state=RANDOM_STATE)
log_reg.fit(X_train_scaled, y_train)
y_pred_log = log_reg.predict(X_test_scaled)

# KNN: try k = 1..11 and choose the best k
accuracies = []
for k in range(1, 12):
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(X_train_scaled, y_train)
    accuracies.append(accuracy_score(y_test, knn_temp.predict(X_test_scaled)))
best_k = list(range(1, 12))[int(np.argmax(accuracies))]
```

```
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)
```

```
# Evaluation
def evaluate_model(name, y_true, y_pred):
    return {'Model': name,
            'Accuracy': accuracy_score(y_true, y_pred),
            'Precision': precision_score(y_true, y_pred),
            'Recall': recall_score(y_true, y_pred),
            'F1-Score': f1_score(y_true, y_pred)}

results_df = pd.DataFrame([
    evaluate_model("Logistic Regression", y_test, y_pred_log),
    evaluate_model("K-Nearest Neighbors", y_test, y_pred_knn)]).set_index('Model')
print(results_df)

print(confusion_matrix(y_test, y_pred_log))
print(confusion_matrix(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_log, target_names=['Not Purchased','Purchased']))
print(classification_report(y_test, y_pred_knn, target_names=['Not Purchased','Purchased']))
```

## 7. Output

### 7.1 Dataset Summary and Preprocessing

- Shape of dataset: (30, 4); no missing values; no duplicate rows.
- Class balance: Not Purchased (0) = 19, Purchased (1) = 11.
- Training set shape: (22, 2); testing set shape: (8, 2).
- After scaling, the training features have mean close to 0 and standard deviation close to 1.

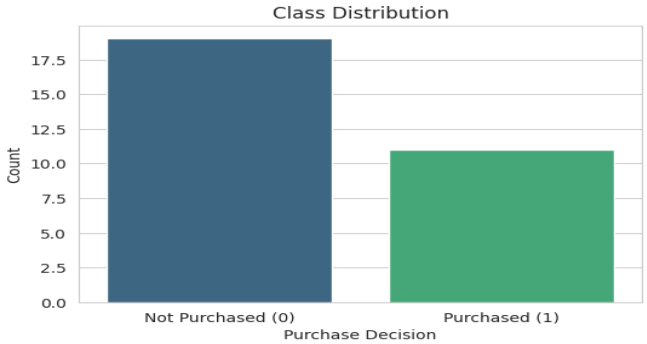



*Figure 1: Class distribution of the target variable*




*Figure 2: Age vs. Estimated Salary by purchase decision*

## 7.2 Choosing the Optimal k for KNN

The best k based on test accuracy was k = 1.

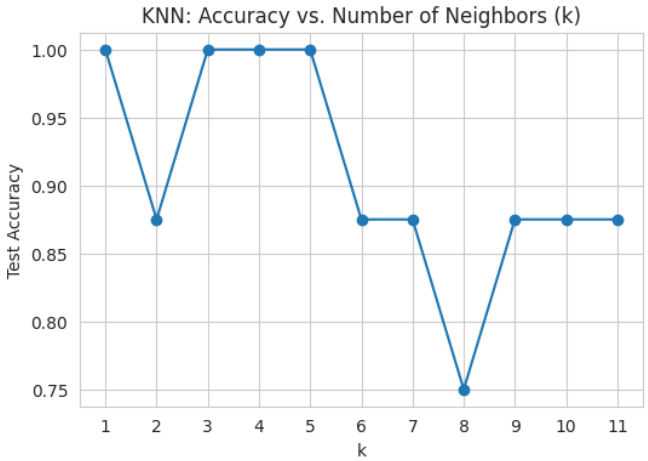



*Figure 3: KNN accuracy vs. number of neighbors (k)*

## 7.3 Performance Metrics

| Model | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Logistic Regression | 0.75 | 0.60 | 1.00 | 0.75 |
| KNN (k = 1) | 1.00 | 1.00 | 1.00 | 1.00 |

## 7.4 Confusion Matrices

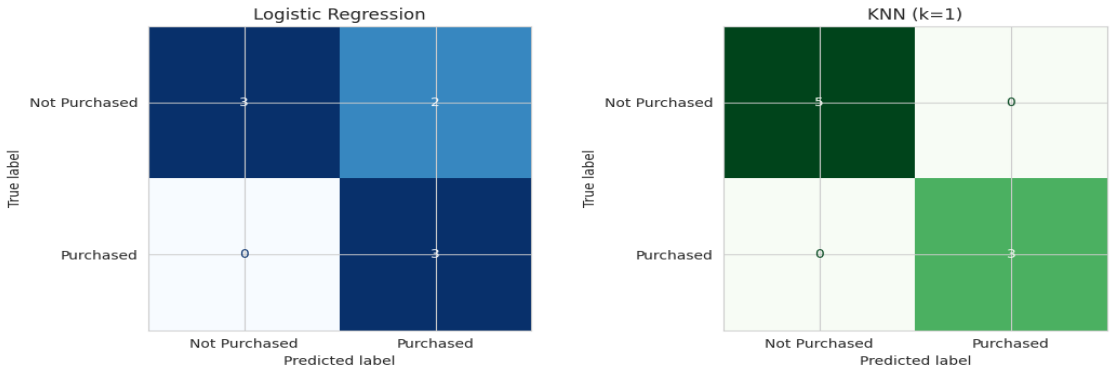



*Figure 4: Confusion matrices for Logistic Regression and KNN*

| Model | TN | FP | FN | TP |
|---|---|---|---|---|
| Logistic Regression | 3 | 2 | 0 | 3 |
| KNN (k = 1) | 5 | 0 | 0 | 3 |

## 7.5 Comparison of Models

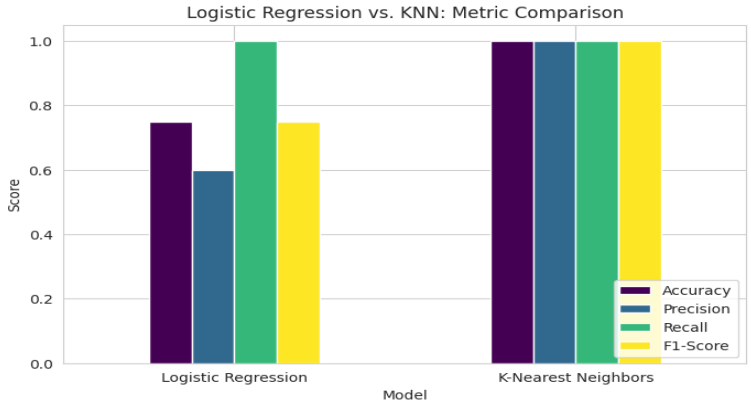



*Figure 5: Metric comparison of Logistic Regression and KNN*

### 8. Result and Analysis

Data preprocessing (missing-value and duplicate check, column selection, stratified train-test split and feature scaling) was carried out on the Social Network Ads dataset, and both classifiers were implemented and evaluated on the 8-record test set.

- **Logistic Regression** reached an accuracy of 0.75. It found all three actual buyers (recall 1.00) but wrongly predicted two non-buyers as buyers, which lowered its precision to 0.60.
- **KNN (k = 1)** classified all 8 test records correctly, giving 1.00 for every metric.
- Logistic Regression can only draw a straight boundary, while KNN adapts to local clusters. This gives KNN an advantage on the mildly curved Age/Salary pattern, at the cost of lower interpretability and sensitivity to k.
- **Important caution:** the test set has only 8 records, so one prediction changes accuracy by 12.5 percentage points. Also, k was chosen using the test accuracy itself, and k = 1 simply

memorizes the nearest training point. The perfect KNN score is therefore optimistic and should not be read as proof that KNN is generally better. A more reliable approach is to choose k with cross-validation on the training data and to use a larger dataset.

**9. Conclusion**

Logistic Regression and KNN were successfully implemented and evaluated for binary classification. Feature scaling is essential for both models, and especially for KNN. On this small dataset KNN scored higher than Logistic Regression, while Logistic Regression offers a simple, interpretable linear boundary. Model choice should consider interpretability, data size and a fair validation method, not the test score alone.

**10. Viva Questions**

1. Define classification. How does it differ from regression? Give two real-world examples of binary classification.
2. Explain the role of the sigmoid function in Logistic Regression. What is the default decision threshold, and what happens to precision and recall if it is lowered to 0.3?
3. Explain the working of the KNN algorithm. What happens to the decision boundary when k is very small and when k is very large?
4. Why is feature scaling necessary for KNN and Logistic Regression? Why must the scaler be fitted only on the training data?
5. For the Logistic Regression confusion matrix obtained in this practical (TN = 3, FP = 2, FN = 0, TP = 3), calculate Accuracy, Precision, Recall and F1-Score by hand and verify them against the program output.
6. In this practical k was chosen using test accuracy. Explain why this is problematic and how k-fold cross-validation solves it. Modify the program to select k using 5-fold cross-validation on the training set and report the new best k and test accuracy.
7. Change random_state (for example 0, 1, 7, 42) and repeat the experiment. Tabulate the accuracy of both models for each seed and comment on the stability of the results.
8. Add the Gender column (after one-hot encoding) as a third feature and observe the effect on both models.
9. Plot the decision boundaries of both models on the scaled Age and Salary axes and explain the visible difference between a linear and a non-linear boundary.
10. Which metric is most important for an ad-targeting problem: precision or recall? Justify your answer with the cost of a false positive and a false negative.

# Assignment No: 07

## PR7: Clustering Techniques (CO4)

Implement:

- K-Means Clustering
- Hierarchical Clustering

Use the Elbow Method and Dendrogram for determining the optimal number of clusters. Analyze the generated clusters using a real-world dataset.

**Aim**

To implement **K-Means Clustering** and **Hierarchical Clustering** on a real-world dataset and determine the optimal number of clusters using the **Elbow Method** and **Dendrogram**. Analyze and compare the generated clusters.

**Theory**

**Clustering**

Clustering is an **unsupervised machine learning** technique that groups similar data points into clusters based on their characteristics. Unlike supervised learning, clustering does not require labeled data.

Applications include:

- Customer segmentation
- Image segmentation
- Market basket analysis
- Document clustering
- Disease classification
- Recommendation systems

**K-Means Clustering**

K-Means is one of the most popular clustering algorithms. It partitions the dataset into **K clusters**, where each data point belongs to the cluster with the nearest centroid.

**Working**

1. Select the number of clusters (K).
2. Randomly initialize K centroids.
3. Assign each data point to the nearest centroid.
4. Recalculate the centroid of each cluster.
5. Repeat Steps 3 and 4 until the centroids no longer change.

**Advantages**

- Simple and easy to implement.
- Fast for large datasets.
- Efficient for spherical clusters.

**Disadvantages**

- Number of clusters (K) must be specified.
- Sensitive to outliers.
- May converge to a local optimum.

**Hierarchical Clustering**

Hierarchical Clustering builds a hierarchy of clusters without requiring the number of clusters initially.
There are two approaches:

- **Agglomerative (Bottom-Up):** Starts with each data point as a separate cluster and merges the closest clusters.
- **Divisive (Top-Down):** Starts with one cluster and splits it recursively.

**Advantages**

- No need to specify K initially.
- Produces a dendrogram for visualization.
- Suitable for small datasets.

**Disadvantages**

- Computationally expensive.
- Difficult to apply to very large datasets.

**Elbow Method**
The Elbow Method is used to determine the optimal number of clusters for K-Means.
It plots:

- Number of clusters (K)
- Within Cluster Sum of Squares (WCSS)

The point where the graph forms an "elbow" indicates the optimal value of **K**.

**Dendrogram**
A Dendrogram is a tree-like diagram used in Hierarchical Clustering.
It shows:

- Cluster merging process
- Similarity between clusters
- Appropriate number of clusters based on the cutting point

**Dataset**
**Dataset Used:** Mall Customer Segmentation Dataset (or any similar real-world dataset)
Features:

- Customer ID
- Gender
- Age
- Annual Income
- Spending Score

**Algorithm**
**Algorithm for K-Means Clustering**
**Step 1:** Start.
**Step 2:** Import required libraries.
**Step 3:** Load the dataset.
**Step 4:** Select numerical features.
**Step 5:** Apply the Elbow Method to determine the optimal number of clusters.
**Step 6:** Train the K-Means model using the optimal K.
**Step 7:** Assign cluster labels to each data point.
**Step 8:** Visualize the clusters.
**Step 9:** Stop.

**Algorithm for Hierarchical Clustering**
**Step 1:** Start.
**Step 2:** Load the dataset.
**Step 3:** Select numerical features.
**Step 4:** Generate the Dendrogram.
**Step 5:** Select the optimal number of clusters.
**Step 6:** Apply Agglomerative Hierarchical Clustering.
**Step 7:** Assign cluster labels.
**Step 8:** Visualize the clusters.
**Step 9:** Stop.

**Pseudocode**
**K-Means Clustering**
BEGIN

Load Dataset

Select Features

FOR K = 1 to 10

    Apply K-Means

    Compute WCSS

END FOR

Plot Elbow Graph

Choose Optimal K

Train Final K-Means Model

Assign Cluster Labels

Display Clusters

END

**Hierarchical Clustering**
BEGIN

Load Dataset

Select Features

Generate Dendrogram

Choose Number of Clusters

Apply Agglomerative Clustering

Assign Cluster Labels

Display Clusters

END

## Code

```
# ================================================================
# Practical: K-Means and Hierarchical Clustering
# Dataset: Mall_Customers.csv
# ================================================================

# ----------------------------
# Step 1: Import Required Libraries
# ----------------------------

import pandas as pd                  # Used for reading and handling datasets
import matplotlib.pyplot as plt      # Used for plotting graphs

# Import K-Means algorithm
from sklearn.cluster import KMeans

# Import Hierarchical Clustering algorithm
from sklearn.cluster import AgglomerativeClustering

# Import Dendrogram functions
import scipy.cluster.hierarchy as sch

# ----------------------------------------------
# Step 2: Load Dataset
# ----------------------------------------------

# Read CSV file
dataset = pd.read_csv("Mall_Customers.csv")

# Display first five records
print(dataset.head())

# ----------------------------------------------
# Step 3: Select Features
# ----------------------------------------------

# Selecting only Annual Income and Spending Score
# because they are commonly used for customer segmentation

X = dataset.iloc[:, [3, 4]].values
# ----------------------------------------------
```

```
# Step 4: Find Optimal Clusters using Elbow Method
# ------------------------------------------------
# Empty list to store WCSS values
wcss = []

# Try cluster values from 1 to 10
for i in range(1, 11):

    # Create KMeans model
    kmeans = KMeans(
        n_clusters=i,
        init='k-means++',
        random_state=42
    )

    # Train model
    kmeans.fit(X)

    # Store WCSS value
    wcss.append(kmeans.inertia_)

# Plot Elbow Graph
plt.figure(figsize=(3,3))
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()

# ------------------------------------------------
# Step 5: Apply K-Means Clustering
# ------------------------------------------------
# Choose 5 clusters based on Elbow graph

kmeans = KMeans(
    n_clusters=5,
    init='k-means++',
    random_state=42
)
# Predict cluster labels
y_kmeans = kmeans.fit_predict(X)
# ------------------------------------------------
# Step 6: Visualize K-Means Clusters
# ------------------------------------------------
plt.figure(figsize=(3,3))

# Cluster 1
plt.scatter(
    X[y_kmeans==0,0],
```

```
    X[y_kmeans==0,1],
    label="Cluster 1"
)

# Cluster 2
plt.scatter(
    X[y_kmeans==1,0],
    X[y_kmeans==1,1],
    label="Cluster 2"
)

# Cluster 3
plt.scatter(
    X[y_kmeans==2,0],
    X[y_kmeans==2,1],
    label="Cluster 3"
)

# Cluster 4
plt.scatter(
    X[y_kmeans==3,0],
    X[y_kmeans==3,1],
    label="Cluster 4"
)

# Cluster 5
plt.scatter(
    X[y_kmeans==4,0],
    X[y_kmeans==4,1],
    label="Cluster 5"
)

# Plot Centroids
plt.scatter(
    kmeans.cluster_centers_[:,0],
    kmeans.cluster_centers_[:,1],
    s=200,
    marker='X',
    label="Centroids"
)

plt.title("K-Means Clustering")

plt.xlabel("Annual Income (k$)")

plt.ylabel("Spending Score")

plt.legend()
```

```
plt.show()

# ----------------------------------------------
# Step 7: Plot Dendrogram
# ----------------------------------------------

plt.figure(figsize=(3,3))

sch.dendrogram(
    sch.linkage(
        X,
        method='ward'
    )
)

plt.title("Dendrogram")

plt.xlabel("Customers")

plt.ylabel("Euclidean Distance")

plt.show()

# ----------------------------------------------
# Step 8: Apply Hierarchical Clustering
# ----------------------------------------------

hc = AgglomerativeClustering(
    n_clusters=5,
    metric='euclidean',
    linkage='ward'
)

# Predict clusters
y_hc = hc.fit_predict(X)

# ----------------------------------------------
# Step 9: Visualize Hierarchical Clustering
# ----------------------------------------------

plt.figure(figsize=(3,3))

plt.scatter(
    X[y_hc==0,0],
    X[y_hc==0,1],
    label="Cluster 1"
)

plt.scatter(
```

```
    X[y_hc==1,0],
    X[y_hc==1,1],
    label="Cluster 2"
)

plt.scatter(
    X[y_hc==2,0],
    X[y_hc==2,1],
    label="Cluster 3"
)

plt.scatter(
    X[y_hc==3,0],
    X[y_hc==3,1],
    label="Cluster 4"
)

plt.scatter(
    X[y_hc==4,0],
    X[y_hc==4,1],
    label="Cluster 5"
)

plt.title("Hierarchical Clustering")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.legend()
plt.show()
# ------------------------------------------------
# Step 10: Display Cluster Labels
# ------------------------------------------------

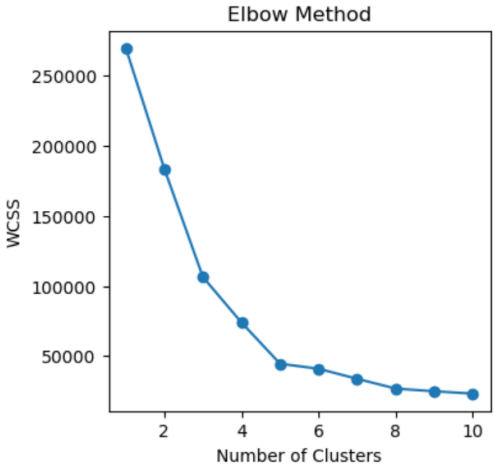
print("\nK-Means Cluster Labels:")
print(y_kmeans)

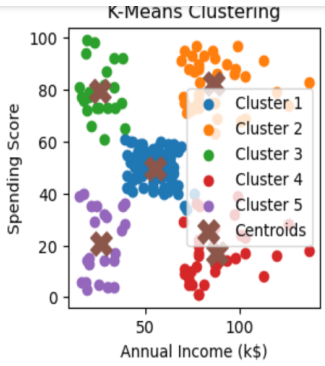
print("\nHierarchical Cluster Labels:")
print(y_hc)

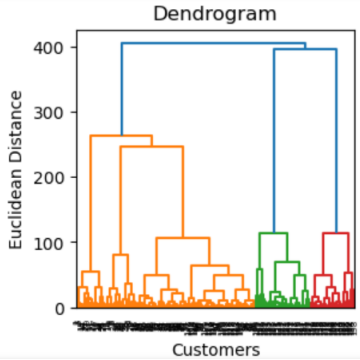
# ------------------------------------------------
# End of Program
# ------------------------------------------------
```

# OutPut

| | CustomerID | Gender | Age | Annual Income (k$) | Spending Score (1-100) |
|---|---|---|---|---|---|
| 0 | 1 | Male | 19 | 15 | 39 |
| 1 | 2 | Male | 21 | 15 | 81 |
| 2 | 3 | Female | 20 | 16 | 6 |
| 3 | 4 | Female | 23 | 16 | 77 |
| 4 | 5 | Female | 31 | 17 | 40 |






```
K-Means Cluster Labels:
[4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4
 2 4 2 4 2 4 0 4 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 3 1 0 1 3 1 3 1 0 1 3 1 3 1 3 1 3 1 0 1 3 1 3 1
 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3
 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1]

Hierarchical Cluster Labels:
[4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4
 3 4 3 4 3 4 1 4 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 2 1 2 1 2 0 2 0 2 1 2 0 2 0 2 0 2 0 2 1 2 0 2 1 2
 0 2 0 2 0 2 0 2 0 2 0 2 1 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 0
 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2 0 2]
```

**Conclusion:** K-Means Clustering and Hierarchical Clustering were successfully implemented on a real-world dataset.

# Assignment No: 08

## PR08: Dimensionality Reduction using Principal Component Analysis (PCA) (CO4)

Apply PCA on a high-dimensional dataset.
Compare:

- Number of features before and after PCA
- Visualization of reduced dimensions
- Impact on classification performance

**Aim:**

**To apply** Principal Component Analysis (PCA) **on a high-dimensional dataset, compare the** number of features before and after PCA**, visualize the reduced dimensions, and analyze the** impact of PCA on classification performance**.**

**Theory**

**Principal Component Analysis (PCA)**

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique used to reduce the number of features in a dataset while preserving as much information (variance) as possible.

PCA transforms the original correlated features into a new set of uncorrelated variables called Principal Components (PCs).

The first principal component captures the maximum variance, the second captures the next highest variance, and so on.

**Why PCA?**

**High-dimensional datasets often contain:**

- Redundant features
- Correlated variables
- Noise
- Increased computational cost

**PCA helps by:**

- Reducing dimensionality
- Improving visualization
- Reducing training time
- Minimizing overfitting
- Preserving important information

**Steps in PCA**

1. Standardize the dataset.
2. Compute the covariance matrix.
3. Calculate eigenvalues and eigenvectors.
4. Sort eigenvalues in descending order.
5. Select the top principal components.
6. Transform the original dataset into the reduced feature space.

**Dataset**

Dataset Used: Breast Cancer Wisconsin Dataset (or any high-dimensional dataset such as Wine or Digits)

**Example:**

- Original Features = 30
- Target Classes = 2 (Benign and Malignant)

**Algorithm**

**Algorithm for PCA**

Step 1: Start.

Step 2: Import the required libraries.

Step 3: Load the high-dimensional dataset.

Step 4: Separate features and target variable.

Step 5: Standardize the feature values.

Step 6: Apply PCA and select the required number of principal components (e.g., 2).

Step 7: Transform the original dataset into the reduced feature space.

Step 8: Train a classifier (e.g., Logistic Regression or SVM) using:

- Original dataset
- PCA-transformed dataset

Step 9: Evaluate both models using Accuracy.

Step 10: Compare the results.

Step 11: Stop.

**Pseudocode**

BEGIN

Load Dataset

Separate Features and Target

Standardize Features

Apply PCA

Select Principal Components

Transform Dataset

Train Classifier on Original Data

Calculate Accuracy

Train Classifier on PCA Data

Calculate Accuracy

Compare Results

Visualize Principal Components

END

**Code:**

```
#  ==========================================
# PCA (Principal Component Analysis)
# ==========================================
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt


# Import dataset
```

```
from sklearn.datasets import load_breast_cancer


# Split dataset into training and testing

from sklearn.model_selection import train_test_split


# Standardize the data

from sklearn.preprocessing import StandardScaler

# PCA algorithm

from sklearn.decomposition import PCA

# Classification model

from sklearn.linear_model import LogisticRegression

# Evaluation metrics

from sklearn.metrics import accuracy_score

# ------------------------------------------

# Step 1: Load the dataset

# ------------------------------------------

# Load Breast Cancer dataset

data = load_breast_cancer()


# Features (input data)

X = data.data


# Target (output labels)

y = data.target
```

```
# Print original number of features
print("Number of features before PCA:", X.shape[1])
# ------------------------------------------
# Step 2: Split the dataset
# ------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
# ------------------------------------------
# Step 3: Standardize the features
# ------------------------------------------

# PCA works better on scaled data
scaler = StandardScaler()
# Fit scaler on training data
X_train_scaled = scaler.fit_transform(X_train)

# Scale testing data
X_test_scaled = scaler.transform(X_test)
```

```
# ----------------------------------------

# Step 4: Train Logistic Regression

# Without PCA

# ----------------------------------------

model_before = LogisticRegression(max_iter=5000)

model_before.fit(X_train_scaled, y_train)

prediction_before = model_before.predict(X_test_scaled)

accuracy_before = accuracy_score(y_test, prediction_before)


# ----------------------------------------

# Step 5: Apply PCA

# ----------------------------------------

# Reduce data to 2 principal components

pca = PCA(n_components=2)

# Apply PCA on training data

X_train_pca = pca.fit_transform(X_train_scaled)

# Apply same transformation on testing data

X_test_pca = pca.transform(X_test_scaled)

# Print number of features after PCA

print("Number of features after PCA:", X_train_pca.shape[1])

# ----------------------------------------

# Step 6: Train Logistic Regression

# With PCA

# ----------------------------------------
```

```
model_after = LogisticRegression(max_iter=5000)
model_after.fit(X_train_pca, y_train)
prediction_after = model_after.predict(X_test_pca)
accuracy_after = accuracy_score(y_test, prediction_after)
# ----------------------------------------
# Step 7: Display Accuracy
# ----------------------------------------
print("\nAccuracy Before PCA :", round(accuracy_before,4))
print("Accuracy After PCA  :", round(accuracy_after,4))
# ----------------------------------------
# Step 8: Visualize PCA Result
# ----------------------------------------
plt.figure(figsize=(4,4))
# Scatter plot of first two principal components
plt.scatter(
    X_train_pca[:,0],
    X_train_pca[:,1],
    c=y_train,
    cmap='viridis'
)
plt.title("PCA Visualization (2 Principal Components)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(label="Class")
```

```
plt.show()

# ----------------------------------------

# Step 9: Explained Variance

# ----------------------------------------

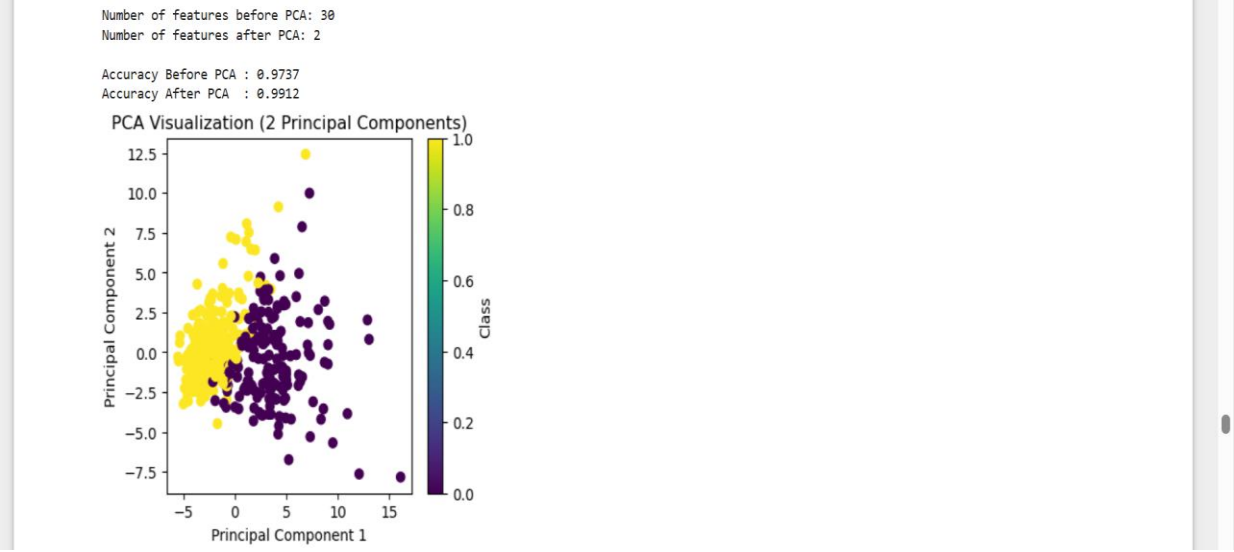
print("\nExplained Variance Ratio:")

print(pca.explained_variance_ratio_)

print("\nTotal Variance Preserved:",)

round(sum(pca.explained_variance_ratio_)*100,2), "%")
```

**Output**




**Conclusion:** Principal Component Analysis (PCA) was successfully applied to a high-dimensional dataset to reduce the number of features while retaining most of the important information.

# Assignment No: 09

## PR11: Advanced Machine Learning and Model Optimization (CO5)

Implement and compare:

- Support Vector Machine (SVM)
- Random Forest
- AdaBoost

Perform:

- Cross Validation
- Hyperparameter Tuning using Grid Search

Evaluate and compare model performance before and after optimization.

**Aim:**

To implement Support Vector Machine (SVM), Random Forest, and AdaBoost classifiers on a suitable dataset, perform Cross Validation and Hyperparameter Tuning using Grid Search, and compare the performance of the models before and after optimization.

**Theory**

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression. It finds the optimal hyperplane that maximizes the margin between different classes.

**Advantages**

- Effective for high-dimensional data.
- Performs well with clear class separation.
- Robust against overfitting with proper parameter tuning.

**Disadvantages**

- Computationally expensive for large datasets.
- Sensitive to parameter selection.

**Random Forest**

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees using the bagging technique. The final prediction is made by majority voting.

**Advantages**

- High accuracy.
- Reduces overfitting.
- Handles missing values and noisy data.

**Disadvantages**

- Higher memory usage.
- Less interpretable than a single Decision Tree.

**AdaBoost**

AdaBoost (Adaptive Boosting) is an ensemble learning algorithm that combines multiple weak learners to build a strong classifier. Each new learner focuses more on previously misclassified samples.

**Advantages**

- Improves classification accuracy.
- Reduces bias.

- Performs well on moderately complex datasets.

**Disadvantages**

- Sensitive to noisy data and outliers.
- May require careful parameter tuning.

**Cross Validation**

Cross Validation is used to evaluate model performance more reliably by dividing the dataset into multiple folds.

**K-Fold Cross Validation**

1. Divide the dataset into K equal folds.
2. Train the model on K−1 folds.
3. Test it on the remaining fold.
4. Repeat until each fold has been used for testing.
5. Compute the average performance.

**Advantages**

- Better estimation of model performance.
- Reduces overfitting.
- Uses the dataset efficiently.

**Hyperparameter Tuning (Grid Search)**

Grid Search systematically searches through a predefined set of hyperparameter values to identify the combination that yields the best model performance.

Example hyperparameters:

**SVM**

- Kernel
- C
- Gamma

**Random Forest**

- Number of Trees (n_estimators)
- Maximum Depth (max_depth)
- Minimum Samples Split (min_samples_split)

**AdaBoost**

- Number of Estimators
- Learning Rate

**Dataset**

Dataset Used: Breast Cancer Wisconsin Dataset (or Iris, Heart Disease, etc.)

Target Classes:

- Benign
- Malignant

**Algorithm**

Step 1

Import the required libraries.

Step 2

Load the dataset.

Step 3

Preprocess the data if required.

Step 4

Split the dataset into training and testing sets.
Step 5
Train SVM, Random Forest, and AdaBoost using default parameters.

Step 6
Evaluate the models.
Step 7
Perform K-Fold Cross Validation.
Step 8
Apply Grid Search to find the best hyperparameters.
Step 9
Retrain the models using the optimized parameters.
Step 10
Evaluate the optimized models.
Step 11
Compare the performance before and after optimization.
Step 12
Stop.

**Pseudocode**
BEGIN

Load Dataset

Split Dataset

Train SVM

Train Random Forest

Train AdaBoost

Evaluate Models

Perform Cross Validation

Perform Grid Search

Retrain Optimized Models

Evaluate Optimized Models

Compare Results

END

# Code:

```
# ============================================================
```

```
# AIM:
# Implement and compare:
# 1. Support Vector Machine (SVM)
# 2. Random Forest
# 3. AdaBoost
#
# Perform:
# • Cross Validation
# • Hyperparameter Tuning using Grid Search
# • Compare performance before and after optimization
# ============================================================

# ----------------------------
# Import required libraries
# ----------------------------
import pandas as pd

# Load Breast Cancer dataset
from sklearn.datasets import load_breast_cancer

# Split dataset into training and testing sets
from sklearn.model_selection import train_test_split

# Standardize features
from sklearn.preprocessing import StandardScaler

# Machine Learning models
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier

# Cross Validation
from sklearn.model_selection import cross_val_score

# Grid Search for Hyperparameter Tuning
from sklearn.model_selection import GridSearchCV

# Evaluation metrics
from sklearn.metrics import accuracy_score, classification_report

# ------------------------------------------------
# Step 1: Load Dataset
# ------------------------------------------------
data = load_breast_cancer()

# Input features
X = data.data

# Output labels
y = data.target
```

```
# ------------------------------------------------
# Step 2: Split Dataset
# ------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# ------------------------------------------------
# Step 3: Feature Scaling
# (Important for SVM)
# ------------------------------------------------
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ================================================
# PART 1 : BEFORE OPTIMIZATION
print("\n========== BEFORE OPTIMIZATION ==========\n")
# Dictionary of models
models = {
    "SVM": SVC(),
    "Random Forest": RandomForestClassifier(random_state=42),
    "AdaBoost": AdaBoostClassifier(random_state=42)
}
# Store accuracies
before_accuracy = {}
# Train and evaluate each model
for name, model in models.items():

    # Train model
    model.fit(X_train, y_train)

    # Predict test data
    prediction = model.predict(X_test)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, prediction)

    # Store accuracy
    before_accuracy[name] = accuracy

    print(name)
    print("Accuracy:", round(accuracy,4))
```

```
    # Perform 5-Fold Cross Validation
    cv_score = cross_val_score(
        model,
        X_train,
        y_train,
        cv=5
    )

    print("Cross Validation Accuracy:",
          round(cv_score.mean(),4))

    print("-----------------------------------")
# PART 2 : GRID SEARCH (HYPERPARAMETER TUNING)
print("\n========== AFTER OPTIMIZATION ==========\n")

# SVM Grid Search
# ----------------------------
svm_parameters = {
    'C':[0.1,1,10],
    'kernel':['linear','rbf']
}

svm_grid = GridSearchCV(
    SVC(),
    svm_parameters,
    cv=5
)
svm_grid.fit(X_train,y_train)
best_svm = svm_grid.best_estimator_

# ----------------------------
# Random Forest Grid Search
# ----------------------------
rf_parameters = {
    'n_estimators':[50,100],
    'max_depth':[5,10,None]
}

rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    rf_parameters,
    cv=5
)

rf_grid.fit(X_train,y_train)

best_rf = rf_grid.best_estimator_

# ----------------------------
```

```
# AdaBoost Grid Search
# ----------------------------
ada_parameters = {
    'n_estimators':[50,100],
    'learning_rate':[0.5,1.0]
}

ada_grid = GridSearchCV(
    AdaBoostClassifier(random_state=42),
    ada_parameters,
    cv=5
)

ada_grid.fit(X_train,y_train)

best_ada = ada_grid.best_estimator_

# Dictionary of optimized models
optimized_models = {
    "SVM": best_svm,
    "Random Forest": best_rf,
    "AdaBoost": best_ada
}
after_accuracy = {}
# ----------------------------------------------------
# Evaluate optimized models
# ----------------------------------------------------
for name, model in optimized_models.items():

    # Train optimized model
    model.fit(X_train,y_train)

    # Predict
    prediction = model.predict(X_test)

    # Accuracy
    accuracy = accuracy_score(y_test,prediction)
    after_accuracy[name] = accuracy
    print(name)

    print("Best Parameters:")
    if name=="SVM":
        print(svm_grid.best_params_)
    elif name=="Random Forest":
        print(rf_grid.best_params_)
    else:
        print(ada_grid.best_params_)

    print("Optimized Accuracy:",
```

```
        round(accuracy,4))

    print("Classification Report")
    print(classification_report(y_test,prediction))
    print("-----------------------------------")

# ====================================================
# FINAL COMPARISON
# ====================================================

print("\n========== FINAL COMPARISON ==========\n")

comparison = pd.DataFrame({
    "Model": before_accuracy.keys(),
    "Before Optimization": before_accuracy.values(),
    "After Optimization": after_accuracy.values()
})

print(comparison)
```

## Output

```
========== BEFORE OPTIMIZATION ==========

SVM
Accuracy: 0.9825
Cross Validation Accuracy: 0.9758
-----------------------------------
Random Forest
Accuracy: 0.9649
Cross Validation Accuracy: 0.9582
-----------------------------------


AdaBoost
Accuracy: 0.9737


Cross Validation Accuracy: 0.9648
-----------------------------------

========== AFTER OPTIMIZATION ==========


    SVM
    Best Parameters:
    {'C': 1, 'kernel': 'rbf'}
    Optimized Accuracy: 0.9825
    Classification Report
                  precision    recall  f1-score   support

               0       1.00      0.95      0.98        43
               1       0.97      1.00      0.99        71

        accuracy                           0.98       114
       macro avg       0.99      0.98      0.98       114
    weighted avg       0.98      0.98      0.98       114

    -----------------------------------


  Random Forest
  Best Parameters:
  {'max_depth': 5, 'n_estimators': 100}
  Optimized Accuracy: 0.9649
  Classification Report
                precision    recall  f1-score   support

             0       0.98      0.93      0.95        43
             1       0.96      0.99      0.97        71

      accuracy                           0.96       114
     macro avg       0.97      0.96      0.96       114
  weighted avg       0.97      0.96      0.96       114

  -----------------------------------
```

```
AdaBoost
Best Parameters:
{'learning_rate': 1.0, 'n_estimators': 100}
Optimized Accuracy: 0.9737
Classification Report
              precision    recall  f1-score   support

           0       0.98      0.95      0.96        43
           1       0.97      0.99      0.98        71

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114

-----------------------------------
```

```
========== FINAL COMPARISON ==========

           Model  Before Optimization  After Optimization

========== FINAL COMPARISON ==========

           Model  Before Optimization  After Optimization
0            SVM             0.982456            0.982456
1  Random Forest             0.964912            0.964912
2       AdaBoost             0.973684            0.973684
```

**Conclusion:** Support Vector Machine (SVM), Random Forest, and AdaBoost classifiers were successfully implemented and compared.

# Assignment No: 10

## PR12: AI/ML-Based Real-World Application (CO1–CO5)

- Disease Prediction System
- Fraud Detection System
- Customer Segmentation
- Recommendation System
- Student Performance Prediction
- Crop Yield Prediction

**The project should include:**

- Problem Definition
- Dataset Collection
- Data Preprocessing
- Model Development
- Performance Evaluation
- Model Optimization
- Result Analysis and Documentation